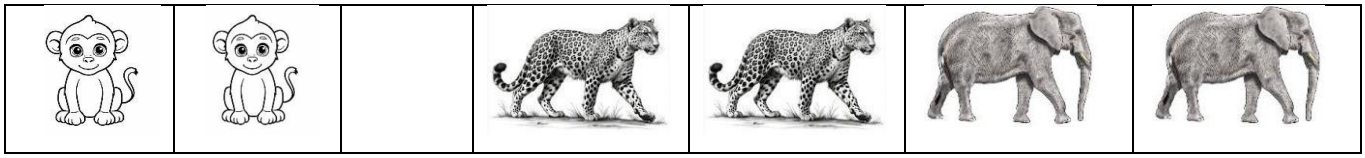
# Assignment 01

## Task 01:

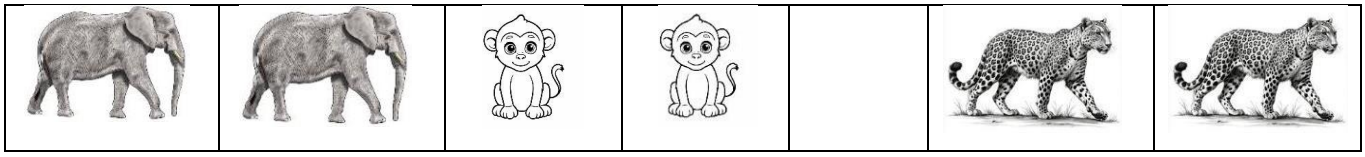
A group of jungle animals needs to cross a narrow bridge in the middle of a dense jungle.

- *2 Baboons (B)*
- *2 Leopards (L)*
- *2 Elephants (E)*
- *1 Empty Space (-)*

The initial arrangement of the animals on the bridge is as follows:



The goal state is:



**Rules:**

1. An animal can move into an adjacent empty space.
2. An animal can also jump over one or two animals (of any type) into the empty space.
3. The empty space (E) must remain between the animals at all times, facilitating their movements.

**Outline** an algorithm to solve this problem and implement it in C++. Additionally, display the complete series of moves required to position the elephants on the left, the baboons in the middle, and the leopards on the right, while strictly following the given movement rules.

## Code:

```
#include <iostream>
#include <queue>
#include <set>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

class State{
private:
    vector<char> s;

public:
    State() {}

    State(vector<char> s){
        this->s = s;
    }

    vector<char> getState(){
        return s;
    }

    void display(){
        for(char c : s) cout << c << " ";
        cout << "\n";
    }

    bool isEqual(State goal){
        return s == goal.getState();
    }

    int findEmpty(){
        for(int i = 0; i < s.size(); i++){
            if(s[i] == '-') return i;
        }
        return -1;
    }

    void swapAnimal(int i, int j){
        swap(s[i], s[j]);
    }

    string toString(){
        return string(s.begin(), s.end());
```

```
    }

};

class Node{
public:

    State state;
    Node* parent;

    Node(State state, Node* parent = nullptr){
        this->state = state;
        this->parent = parent;
    }
};

class BridgeSolver{
private:
    State start;
    State goal;

public:
    BridgeSolver(State start, State goal){
        this->start = start;
        this->goal = goal;
    }

    // after solving, we'll print path
    void printPath(Node* node){
        vector<State> path;

        while(node != nullptr){
            path.push_back(node->state);
            node = node->parent;
        }

        reverse(path.begin(), path.end());

        for(int i = 0; i < path.size(); i++){
            cout << "Move " << i << ": ";
            path[i].display();
        }
        cout << "\n";
    }

    void solve(){
```

```
        queue<Node*> q;
        set<string> visited;

        Node* root = new Node(start);

        q.push(root);
        visited.insert(start.toString());

        while(!q.empty()){
            Node* current = q.front();
            q.pop();

            if(current->state.isEqual(goal)){
                printPath(current);
                return;
            }

            int empty = current->state.findEmpty();
            int size = current->state.getState().size();

            vector<int> moves = {-3, -2, -1, 1, 2, 3};

            for(int move : moves){
                int newPos = empty + move;

                if(newPos < 0 || newPos >= size){
                    continue;
                }

                State child = current->state; // copying the parent into child

                child.swapAnimal(empty, newPos); // updating the child

                string key = child.toString(); // doing this to check it in set

                if(visited.find(key) == visited.end()){
                // visited.end() = the last imaginery position (if set doesn't finds
hey, it returns this..)

                    visited.insert(key);

                    Node* childNode = new Node(child, current);

                    q.push(childNode);
                }
            }
```

```
        }

        cout << "No solution found.\n";
    }
};


int main(){
    vector<char> initial = {'B', 'B', '-', 'L', 'L', 'E', 'E'};
    vector<char> goal = {'E', 'E', 'B', 'B', '-', 'L', 'L'};

    State start(initial);
    State end(goal);

    BridgeSolver solver(start,end);

    solver.solve();
}
```

**Output:**

```
Move 0: B B - L L E E
Move 1: B B E L L - E
Move 2: B B E L - L E
Move 3: B B E L E L -
Move 4: B B E - E L L
Move 5: - B E B E L L
Move 6: E B - B E L L
Move 7: E - B B E L L
Move 8: E E B B - L L
```

## Task 02:

You are playing Spiral Vault Escape. A 3×3 vault floor contains numbered energy tiles. To unlock the vault, you must step on every tile exactly once in a clockwise spiral pattern, starting from the top-left corner.

Movement pattern:

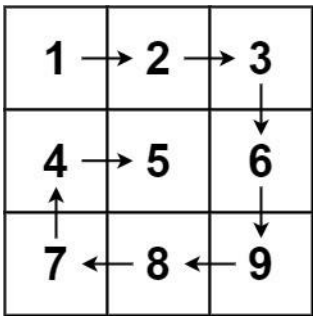
1. Move right → Move down ↓ Move left ← Move up ↑
2. Repeat until all tiles are activated.

Game Rules

- Start at position `(0,0)` → tile `1`
- You cannot revisit a tile.
- You must always continue straight until blocked.
- When blocked, turn clockwise.
- Stop when all 9 tiles are activated.

If done correctly, the tiles activate in this exact order:

```
[1, 2, 3, 6, 9, 8, 7, 4, 5]
```




Outline a solution for this game that outputs the activation sequence of tiles in the correct spiral order. Show the complete sequence of tile activations as output when your program runs.

## Code:

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> spiralOrder(vector<vector<int>> a){
    vector<int> ans;

    int m = a.size();
    int n = a[0].size();
```

```
    int right = n-1;
    int bottom = m-1;
    int left = 0;
    int up = 0;

    while(up <= bottom && right >= left){
        // left ---> right
        for(int i = left; i <= right; i++){
        //           ^from where we are starting

            ans.push_back(a[up][i]);
            //                        ^on which line we are moving
        }
        up++; // becuase we have moved in this line

        // up ---> bottom
        for(int i = up; i <= bottom; i++){
            ans.push_back(a[i][right]);
        }
        right--;

        // right ---> left
        if(up <= bottom){
            for(int i = right; i >= left; i--){
                ans.push_back(a[bottom][i]);
            }
            bottom--;
        }

        // bottom ---> up
        if(left <= right){
            for(int i = bottom; i >= up; i--){
                ans.push_back(a[i][left]);
            }
            left++;
        }
    }
    return ans;
}

int main(){
    vector<vector<int>> matrix = {
        {1,  2, 3, 4},
        {5,  6, 7, 8},
        {9, 10,11,12},
```

```
        {13,14,15,16}
    };

    vector<int> result = spiralOrder(matrix);

    cout << "Spiral Order: ";

    for(int i = 0; i < result.size(); i++) cout << result[i] << " ";
    cout << "\n";
}
```

**Output:**

```
Spiral Order: 1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```

MUHAMMAD HASAN 02-135251-040

# Data Structures & Algorithms

Date:

## Assignment 01

Question 3:

| Algorithm | Best Case | Average Case | Worst Case | Stable | Suitability |
|---|---|---|---|---|---|
| Bubble | $O(n)$ | $O(n^2)$ | $O(n^2)$ | Yes | Small or nearly sorted |
| Selection | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | NO | When we don't want too much swap |
| Insertion | $O(n)$ | $O(n^2)$ | $O(n^2)$ | Yes | Small or nearly sorted |
| Merge | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | Yes | Large dataset & when stability is required. |
| Quick | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | No | Large dataset |

→ Merge sort is the recommendation in this scenario, since the dataset is large so even worst case there will be guranteed $O(n \log n)$ time complexity. Also this is stable.

Date: _____

Question 4:

Since, the input size (n) & range (k) is given. Counting sort will be the best choice for this scenario. It will take time complexity of O(n+k) which is even better than merge sort.
Also it uses extra space of O(k).

Question 5:

Here, the data is sorted, & after each check we'll be informed if the required value is high or low.
This is a perfect condition for Binary Search. Also it has time complexity of O(log₂n).
In this scenario
n = 1000,000

$\log_2(1000000) \approx 20$ so this also fulfills our life line criteria...